

Informatik BW: Python Crash Course A

VU Informatik BW, WS24

Thorsten Ruprechter

Wiederholung

- LV-Ablauf
- Bewertungsmodus
- Termine
- Organisatorisches
- Motivation & Erste Grundbegriffe
- Python & Hello World
- Ausgabe Assignment 0

Kurze Wiederholung: Einführung

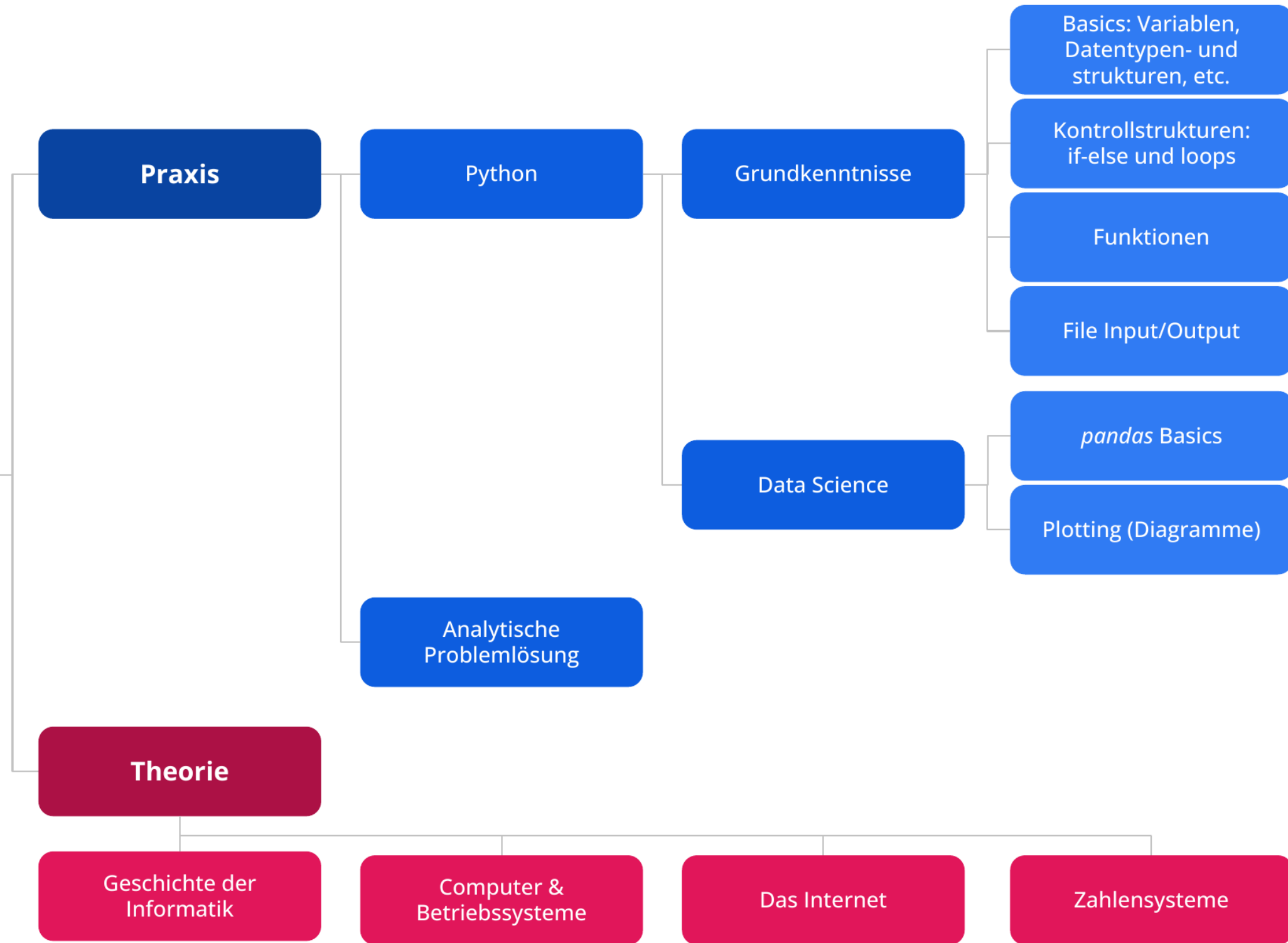


fbr.io/infobw1

Zeit: 2 Minuten



- Variablen
- Integers, Floats und Bool'sche Werte
- Operatoren
- Strings
- Casting
- None-Keyword
- Bedingungen (if-Statements)
- Erinnerung: Assignment 0



Variablen (0)

- Label (Etikett) für Werte oder Berechnungen
 - „Variable referenziert bestimmten Wert“
 - „Platzhalter“
- Zuweisung von Werten mit `=`
 - `<Variablenname> = <Zugewiesener Wert>`
- Vorübergehendes Speichern von Werten
 - „Lebensdauer“ bis Programmende
- Startwert notwendig, dann beliebig änderbar

```
number    = 123  
message   = 'Hello World!'  
number2   = number
```

Variablen (1)

- Benennungsvorschriften für Variablennamen
 - Fixe Regeln (Missachtung führt zu Fehlern)
 - Nur Buchstaben, Zahlen, „Underscore“
 - Keine Zahl am Beginn
 - Keine Leerzeichen
 - Guidelines (stylistisch)
 - Lower case, kurz aber aussagekräftig
 - Wörter getrennt durch „Underscore“ (_)

Variablen: Erstes Code-Beispiel

siehe Code-Beispiele

Datentypen für Variablen (Auswahl)

Datentyp	Funktion	Beispiele
<code>int</code>	ganze Zahlen	<code>x = 3</code>
<code>float</code>	Fließkommazahlen	<code>x = 3.0</code>
<code>bool</code>	Boolesche Werte	<code>x = True</code> <code>x = False</code>
<code>str</code>	Strings, Zeichenketten	<code>x = 'Test'</code> <code>x = "Test"</code>
<code>tuple</code>	Tupel	<code>x = (1, 2, 3)</code>
<code>list</code>	Liste	<code>x = [1, 2, 3]</code>
<code>dict</code>	Dictionary	<code>x = {1: 'blue', 2: 'red'}</code>
<code>io.TextIOWrapper</code>	Dateien	<code>x = open('readme.txt')</code>
<code>...</code>	<code>...</code>	<code>...</code>

Tabelle 1: Kurze Übersicht einer Auswahl von Datentypen in Python.

Integer & Floating Point Numbers (0): Ganzzahlen und Gleitkommazahl

- Integer (Ganzzahlen):
 - 1, -13, 15, -42, 12345, ...
- Float (Gleitkommazahl)
 - 1.3, 3.14159, -42.123, ...
- Operationen
 - Addieren (+), Subtrahieren (-), Multiplizieren (*), Dividieren (/)
 - Exponieren (**)
 - Modulo Operator (%)

```
number = 5 + 13  
number_2 = number + 5
```

Integer & Floating Point Numbers (1): Erweiterte Operatoren

- Grundrechnung mit direkter Zuweisung:
 - Addition (`+=`), Subtraktion (`-=`), Multiplikation (`*=`), Division (`/=`)

```
number = 42
```

```
number += 8
```

```
number -= 10
```

```
number /= 4
```

```
number *= 2
```

```
number = 42
```

```
number = number + 8
```

```
number = number - 10
```

```
number = number / 4
```

```
number = number * 2
```

Integer & Floating Point Numbers (2): Details und Besonderheiten

- Operation mit Integer und Float = Float
 - `4/2 = 2.0`, `2 * 3.0 = 6.0`, `3.0 ** 2 = 9.0`
- Underscores in Zahlen möglich zur Formatierung
`universe_age = 14_000_000_000`
 $\Leftrightarrow$
`universe_age = 14000000000`

Integer, Floats, und erste Operatoren: Code

siehe Code-Beispiele

Strings/Zeichenketten (0)

- Text = Folge von Zeichen („Characters“)
- Start und Ende gekennzeichnet mit Anführungszeichen (" oder ')

```
string = 'Das ist ein String'
```

```
string = "Das ist auch ein String"
```

```
string = '"Python" ist nach "Monty Python" benannt.'
```

```
string = '''Dies ist ein  
           mehrzeiliger String'''
```

```
string = """Dies ist auch ein  
           mehrzeiliger String"""
```

Strings/Zeichenketten (1)

- Zugriff auf...
 - einzelne Zeichen: `[]` und Index (0-basiert!)
 - Erstes Zeichen = Index 0
 - Letztes Zeichen = Länge von String - 1
 - mehrere Zeichen („Slicing“): `[], :` und Indices
 - Länge: `len(string)`

Strings (2): Zugriff auf Teil-Strings (Slicing)

■ Zugriffsarten

- `string[n]`

➔ `n`-tes Zeichen (`n` = Zahl)

- `string[start:ende]`

➔ Zeichen von `start` (inklusive) bis `ende` (exklusive)

- `string[start:ende:schrittweite]`

➔ Zeichen von `start` (inklusive) bis `ende` (exklusive)
in Intervall (`schrittweite`)

Strings (3): Zugriff auf Teil-Strings (Slicing)

- Zugriffsarten (fortgeschritten)
 - `string[-n]`
→ `n`-t letztes Zeichen (`n` = Zahl)
 - `string[-n:]`
→ alles von (inklusive) `n`-t letztem Zeichen
 - `string[:-n]`
→ alles bis (exklusive) `n`-t letztem Zeichen
 - `string[::-1]`
→ String in umgekehrter Reihenfolge

Strings (4): Slicing-Übung

```
text = 'TUGraz'
```

Befehl mit text	Resultat
	'T'
text[:1]	
text[6]	
	'z'
	'Graz'
text[::-2]	
	zarGUT

Beispiel: String slicing

Strings (5): Slicing-Übung

```
text = 'TUGraz'
```

Befehl mit text	Resultat
text[0]	'T'
text[:2] text[0:2]	'TU'
text[6]	IndexError
text[5] text[-1]	'z'
text[2:] text[2:6]	'Graz'
text[::2]	TGa
text[::-1]	zarGUT

Lösung: String slicing

Strings/Zeichenketten (6): Simple Methoden

- Flexibel, große Menge an fertig eingebauter Funktionalität
- Methode = Aktion auf Wert (Objekt) in Variable
- Ausgangsstrings (Tabelle):

```
name = 'max muster'
text = ' Python! '
```

Aufruf	Resultat
name + ' , hi!'	'max muster, hi!'
text * 2	' Python! Python! '
name.title()	'Max Muster'
name.upper()	'MAX MUSTER'
text.lower()	' python! '
text.rstrip()	' Python!'
text.lstrip()	'Python! '
text.strip()	'Python!'
name.split(' ')	['max', 'muster']

Tabelle 7: String Funktionalität und Methoden

<https://docs.python.org/3/library/stdtypes.html#string-methods>

Strings: Code

siehe Code-Beispiele

Datentypen umwandeln: Casting

- Umwandeln von Wert in anderen Datentyp
 - `int()`, `str()`, `float()`, `bool()`...
- Beispiele
 - `int('5')` $\Rightarrow$ `5`
 - `float('5.23')` $\Rightarrow$ `5.23`
 - `str(5)` $\Rightarrow$ `'5'`
 - `int(True)` $\Rightarrow$ `1`
 - `bool(0)` $\Rightarrow$ `False`

```
num1 = 123
num2 = '15'
num3 = num1 + num2
```

Output:

```
TypeError
  1 num1 = 123
  2 num2 = '15'
--> 3 num3 = num1 + num2
TypeError: unsupported operand
type(s) for +: 'int' and 'str'
```

Lösung:

```
num1 = 123
num2 = '15'
num3 = num1 + int(num2)
```

Kommentare

- Syntax
 - `#` Single-line Comment
 - `"""` Multi-line
Comment `"""`
 - `'''` Multi-line
Documentation `'''`
- Gedanken zu Programmablauf und Struktur
- Viele > Wenige
- Hilfreich für Gruppenprojekte, mehrtägige Arbeiten, etc.

Formatierter Output (0)

- `print()` Funktion
 - Zeilenumbruch am Ende (konfigurierbar)
 - Argumente getrennt durch `,` = Leerzeichen als Abstand
- Ausgabe und Formatierung von Variablen
 - `f` vor String
 - Variable in `{}`

```
name = 'Max'
```

```
age = 20
```

```
print(f'My name is {name} and I am {age} years old')
```

Output: My name is Max and I am 20 years old

Formatierter Output (1)

- Andere Möglichkeiten zur Formatierung
 - format Methode
 - % und Parameter Tupel
- „Whitespaces“ und Sonderzeichen

```
msg='This is Maria\'s Umbrella'
```

Sequenz	Bedeutung
\n	Zeilenumbruch
\t	Tabulator
\'	' Zeichen in Strings
\"	" Zeichen in Strings
\\	das Zeichen \

Tabelle 8: Sonderzeichen in Python

Casting und Formatierter Output: Code

siehe Code-Beispiele

None

- Kein Wert, „null value“, Platzhalter
- Variablenwert nicht festgelegt
- `x = None`, nicht dasselbe wie:
 - `0`
 - `''`
 - `False`

Boolesche Ausdrücke (`boolean`, `bool`)

- Benannt nach George Boole
- „Speichert“ Wahrheitswerte
 - Mensch: Wahr oder Falsch
 - Computer: 1 oder 0
 - **Python:** `True` oder `False`

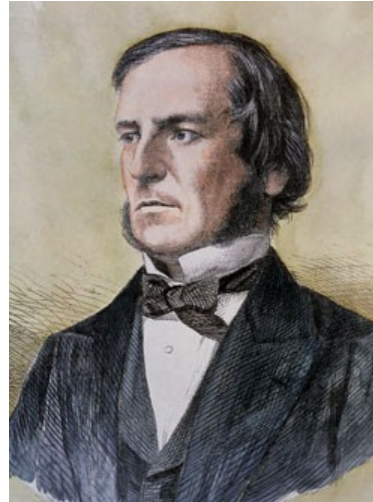


Bild 1: George Boole.
https://upload.wikimedia.org/wikipedia/commons/c/c0/George_Boole_color.jpg

```
value = ((2 + 2) < 4)
print(f'This is {value}')
```

Vergleichsoperatoren

Operator	Beschreibung	Beispiel
<code>==</code>	Ist gleich	<code>x == y</code>
<code>!=</code>	Nicht gleich	<code>x != y</code>
<code>></code>	Größer als	<code>x > y</code>
<code><</code>	Kleiner als	<code>x < y</code>
<code>>=</code>	Größer gleich	<code>x >= y</code>
<code><=</code>	Kleiner gleich	<code>x <= y</code>
<code>is</code>	Beide Variablen sind das gleiche Objekt	<code>x is y</code>
<code>is not</code>	Beide Variablen sind nicht das gleiche Objekt	<code>x is not y</code>

Tabelle 1: Wichtige Vergleichsoperatoren in Python

=> Ergebnis Vergleichsoperatoren: Boolean-Wert (True oder False)

Logische Operatoren (0)

- OR
 - Oder, logische Summe, $+$, $\vee$, ...
- AND
 - Und, logisches Produkt, $*$, $\wedge$, ...
- NOT
 - Verneinung, Komplementierung, $\neg$, $!$, ...

=> Ergebnis logischer Operatoren: Boolean-Wert (True oder False)

Logische Operatoren (1)

Operator	Beschreibung	Beispiel
and	Wahr, wenn beide Statements wahr sind	<code>x == y and y < 5</code>
or	Wahr, wenn eines der Statements wahr ist	<code>x > 2 or y > 3</code>
not	Verneinung des folgenden Vergleichs oder Werts	<code>not x == y</code> <code>not z</code>

Tabelle 2: Logik und Zugehörigkeitsoperatoren

Logische Operatoren (2): Wahrheitstabelle

x	y	x and y
True	True	True
True	False	False
False	True	False
False	False	False

Tabelle 3: AND Wahrheitstabelle

x	y	x or y
True	True	True
True	False	True
False	True	True
False	False	False

Tabelle 4: OR Wahrheitstabelle

x	not x
True	False
False	True

Tabelle 5: NOT Wahrheitstabelle

Exkurs – Operator precedence: Operatoren Auswertungsreihenfolge

- Nicht sicher? Klammern `()`
- Vollständige Liste unter:
 - <https://docs.python.org/3/reference/expressions.html#operator-precedence>

Operator precedence (von niedrigster bis höchster)
or
and
not
in, not in, is, is not, <, <=, >, >=, !=, ==
^
&
<<, >>
+, -
*, @, /, //, %
+X, -X, ~X
**

Tabelle 6: Operatoren Auswertungsreihenfolge

Booleans und Vergleichsoperatoren: Code

siehe Code-Beispiele

Assignment 0 (1 Punkt): “Hello, World!”

Assignment 0 (1)

- Installiere Python (Version ≥ 3.11) und Jupyter Notebooks
 - Hilfe in Präsenz-Tutorien!
- Erstellen von Programm in `.ipynb`-Datei, welches exakt `Hello, World!` ausgibt
- Deadline(s):
 - Soft-Deadline: bis 10.10. 23:59 => max. 1 Punkt
 - Hard-Deadline: bis 11.10. 23:59 => 0 Punkte
 - LV-Abmeldung/"Nicht Bewertet" falls keine Abgabe

Assignment 0 (2)

- Abgabe von Jupyter-Notebook-File mit dem Namen:
 - `assignment0.ipynb`
- Fileinhalt:
 - Inhalt der `assignment0.ipynb`-Template-Datei mit einer neuen Code-Zelle, welche **EXAKT** `Hello, World!` ausgibt
 - Hinweis: siehe vorheriges Beispiel mit `print()`
- Abgabe im TeachCenter

Morgen (11.10.) 9:30 Vorlesung HS P2

VU Informatik BW, WS24

Thorsten Rupprechter